ID NO:2400030040

Name: N. Sri Aravind

#SKILL - 5 ->Spring Autowiring Demo using @Autowired

Prerequisites:

- **General Idea on Spring Framework**
- **Modules of Spring Framework**
- **Understanding of Autowiring concepts**

A college application maintains student information along with their certification details. The Student object contains a Certification object, and instead of creating this Certification object manually, Spring should automatically provide it using the @Autowired annotation. When the required classes are marked with @Component, Spring creates their objects and manages them inside the IoC container. Using @Autowired, the Certification object is supplied to the Student object by Spring without using the new keyword, allowing both objects to be linked through annotation-based configuration.

Tasks:

1. **Create a Certification class with fields such as id, name, and dateOfCompletion.**
2. **Create a Student class with fields such as id, name, gender, and a Certification object.**
3. **Annotate both classes with @Component to enable Spring detection.**
4. **Use @Autowired on a field, constructor, or setter inside the Student class to inject the Certification object.**
5. **Configure component scanning using either: a. An XML configuration file b. A Java configuration class with @Configuration and @ComponentScan**
6. **Load the Spring IoC container using ApplicationContext.**
7. **Retrieve the Student bean and print all details, including the injected Certification object.**
8. **Push the Spring project to GitHub.**

Program Modules:

1)Certification.java

```
package com.klu.model;

import org.springframework.stereotype.Component;

@Component

public class Certification {

   private int id = 201;

   private String name = "Full Stack";

   private String dateOfCompletion = "20-Jan-2026";

   @Override

   public String toString() {

      return "Certification [id=" + id +

            ", name=" + name +

            ", dateOfCompletion=" + dateOfCompletion + "]";

   }

}
```

2)MainApp.java

```
package com.klu.model;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.boot.CommandLineRunner;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.klu.model.Student;

@SpringBootApplication

public class MainApp implements CommandLineRunner {

   @Autowired

   private Student student;


   public static void main(String[] args) {

      SpringApplication.run(MainApp.class, args);

   }
```

```
    @Override
    public void run(String... args) {
        student.displayDetails();
    }
}
```

3)Student.java

```
package com.klu.model;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Student {
    private int id = 101;
    private String name = "Aravind";
    private String gender = "Male";
    @Autowired
    private Certification certification;
    public void displayDetails() {
        System.out.println("Student ID: " + id);
        System.out.println("Student Name: " + name);
        System.out.println("Gender: " + gender);
        System.out.println(certification);
    }
}
```

Output:

```
Student ID: 101
Student Name: Aravind
Gender: Male
Certification [id=201, name=Full Stack, dateOfCompletion=20-Jan-2026]
```

Git Repository:

https://github.com/Aravind2k6/2400030040-Fsad-WorkBook.git

VIVA QUESTIONS:

1. What is autowiring in Spring?

A. Autowiring in Spring is a feature that automatically injects required dependencies into a bean by matching type, name, or constructor, without explicitly configuring them.

2. What is the purpose of the @Autowired annotation?

A. The @Autowired annotation in Spring is used to automatically inject a required dependency into a bean by the Spring container.

3. How does Spring choose which bean to inject?

A. Spring chooses which bean to inject primarily by type**; if multiple beans of the same type exist, it uses @Primary** or **@Qualifier to resolve the ambiguity.**

4. What is component scanning in Spring?

A. Component scanning in Spring is the feature that automatically scans specified packages to detect classes annotated with @Component, @Service, @Repository, and @Controller, and registers them as beans in the Spring container.

5. What happens if more than one bean matches the dependency type?

A. If more than one bean matches the dependency type, Spring throws a NoUniqueBeanDefinitionException unless you resolve it using @Primary or @Qualifier to specify which bean should be injected.